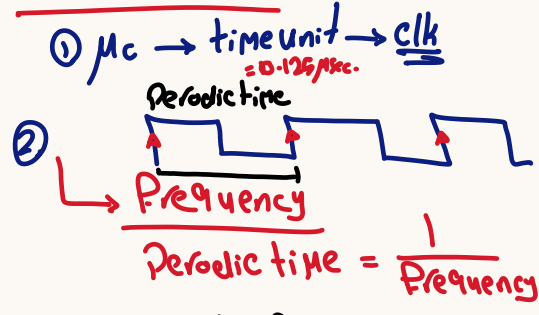


Timer/Counter

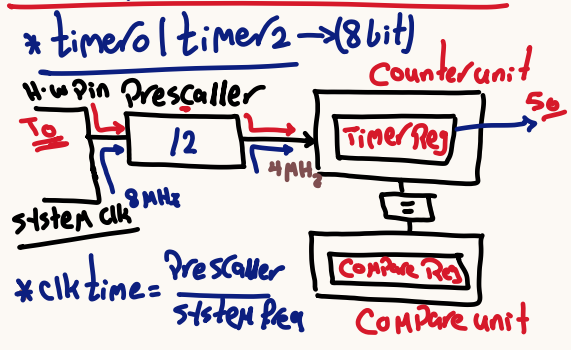
* Some Concept



For ex: system freq = 8 MHz $\rightarrow$ Prescaler = [1]

$$\text{clk time} = \frac{\text{Prescaler}}{\text{system freq}} = \frac{1}{8000000} = 0.125 \mu\text{sec}$$

timer/Counter Peripheral

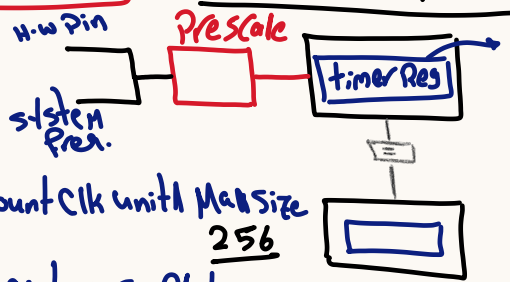


timer Mode

- ① Normal
- ② Compare Match Mode

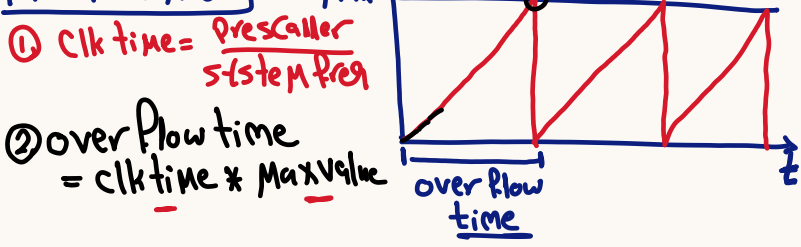
clear time Compare Match $\rightarrow$ PWM Pulse width Modulation

* Normal Mode $\rightarrow$ Over Flow Mode



- * when Register Complete
- set flag $\rightarrow$ over flow flag
 - clear timer Register
 - start from zero

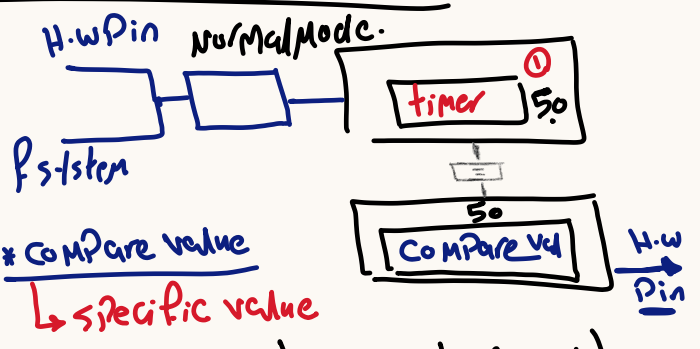
* Normal Mode



①

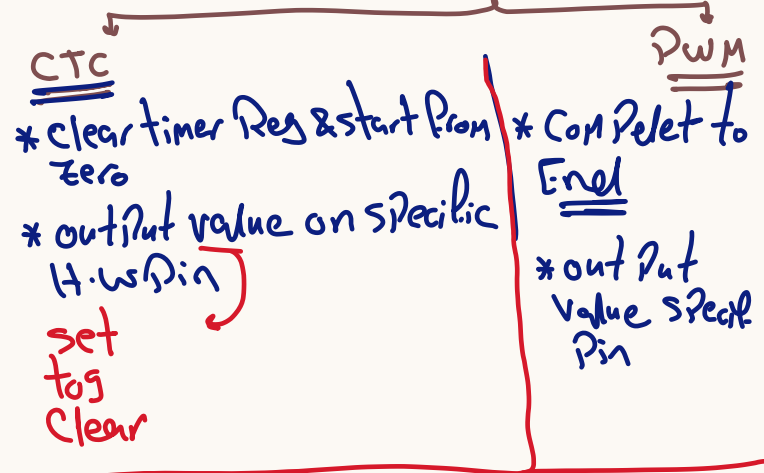
* Compare Match Mode

④

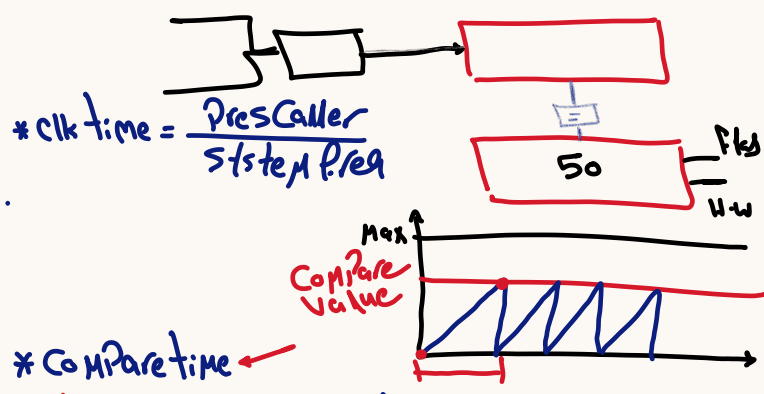


- * Compare operation working all time
- * when Compare Match (timer Reg = Compare value) the timer set flag (Compare Match flag)

* Different between Normal Mode Compare Match $\rightarrow$ Action



CTC Mode $\rightarrow$ Compare value can't start again



* Compare time = $(\text{Compare value}) \times \text{clk time}$

No of CTC Count = $\frac{\text{Requirement time}}{\text{Compare time}}$

- ① Assume $\rightarrow$ No of CTC Count
- $\rightarrow$ Rule $\rightarrow$ No int
 - $\rightarrow$ Compare value can be store in Register
- 300 $\times$ 8bit

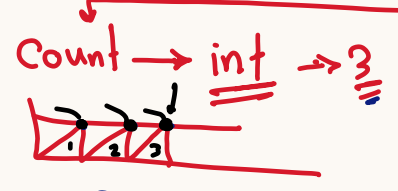
* Relation between Req time & ovf time? (2)

- ① $Req = OVF \rightarrow \checkmark$
- ② $Req > OVF$
- ③ $Req < OVF$

$\rightarrow Req > OVF$

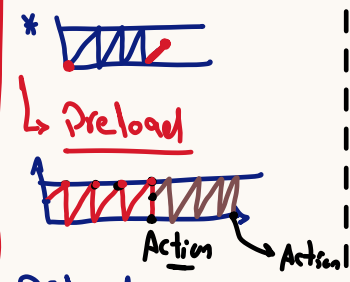
* # No of OVF Count = $\frac{Req\ time}{OVF\ time}$

Note $\rightarrow$ Same unit



```
ISR()
{
    static Count = 0;
    Count++;
    if (Count == 3)
    {
        Action();
        Count = 0;
    }
}
```

count $\rightarrow$ float (3.4)



Preload = $Max\ value * (1 - 0.4)$
 $= 2^n * (1 - 0.4)$
 $= 256 * (1 - 0.4) = 153.6$

Preload $\rightarrow$ timer Register

```
Init()
{
    timer Reg = Pre;
}

ISR()
{
    static Counter = 0;
    Counter++;
    if (Counter == 4)
    {
        Action();
        timer Reg = Pre;
        Counter = 0;
    }
}
```

* Normal Mode

- ① $clk\ time = \frac{Prescaler}{System\ freq}$
- ② $OVF\ time = Max * clk\ time$
 $= 2^n * clk\ time$
- ③ # No of OVF Count = $\frac{Req\ time}{OVF\ time}$
- ④ Pre load value = $\frac{Max * (1 - 0.4)}{2^n * (1 - 0.4)}$
- ⑤ Pre load update in timer Register
- ⑥ in ISR we check with (No of OVF Count + 1)
 $\rightarrow$ in if condition

* $Req < OVF$

No of OVF Count = $\frac{Req}{OVF\ time} = \boxed{\quad}$

Preload value = $2^n * (1 - 0.4)$

ex! toggle led $\rightarrow 1000\ msec$
 time = 8 bit $\rightarrow$ System clk = 8 MHz
 Prescaler = 8
 $clk\ time = \frac{8}{8\ M} = 1\ msec$

No of CTC $\rightarrow 4$

Compare time = $\frac{Req}{No\ of\ CTC} = \frac{1000}{4}$

Compare time = 250 msec

Compare time = $clk\ time * Compare\ value$

Compare value = $\frac{Compare\ time}{clk\ time}$

Compare value = $\frac{250\ msec}{1\ msec} = 250$

PWM ! Pulse width Modulation

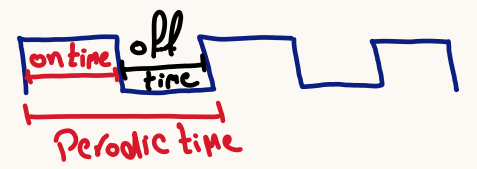
$\rightarrow$ to make mc $\rightarrow$ Analog

DAC

Digital analog convert

Definition

① How mc provide PWM



Periodic time

$\rightarrow$ to make complete clock cycle

Frequency $\rightarrow$ number of cycle in one sec.

on time $\rightarrow$ time $\rightarrow$ High level
 off time $\rightarrow$ " $\rightarrow$ Low level

* Duty cycle ! Ratio between on time & Periodic

Ex: timer 16 bit Fsystem = 8 MHz
 Prescaler 8 Reg = 5sec.

① Clk time = $\frac{8}{8 \text{ MHz}} = 1 \mu\text{sec}.$

② ovf time = $2^{16} * 1 \mu\text{sec} \rightarrow 1024 \mu\text{sec}$

③ No of ovf Count = $\frac{5000000}{1024} = 4882.8125$

④ Preload = $2^{16} * (1 - 0.8125) = 192 \checkmark$

Code

```
init()
{
  timer Reg = 192;
}

ISR()
{
  static Counter = 0;
  if (Counter == 4883)
  {
    Action()
    timer Reg = 192;
    Counter = 0;
  }
}
```

How generate PWM

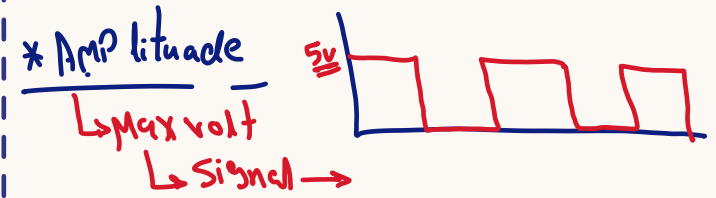
S.W ① delay

```
while(1)
{
  Motor on();
  delay(300);
  Motor off();
  delay(700);
}
```

300 700
 1msec
 Freq = 1 kHz
 Duty cycle = 30%.

③

* Duty Cycle = $\frac{\text{on time}}{\text{Periodic time}} * 100$



* RMS → Root mean squared.

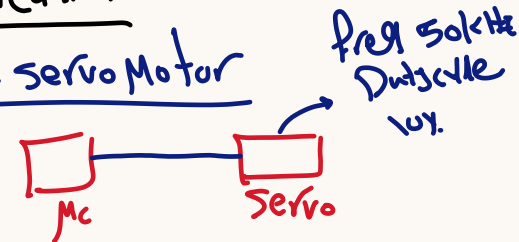
$RMS = A \sqrt{\text{duty cycle}}.$

↳ Analog volt → H.W Component
 with Control

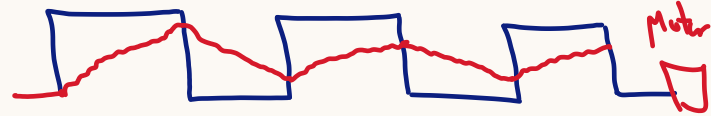
PWM in Embedded

1) Communication

↳ servo Motor



2) RMS (Root Means Squared)



Generate PWM by SW

② Normal Mode / CTC Mode

ISR(ovf) → 10μsec

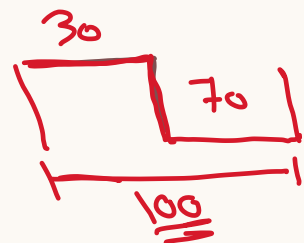
```
{
  static Counter = 0;
  Counter++;
```

```
if (Counter == 30)
```

```
{
  set Pin Low;
```

```
else if (Counter == 100)
```

```
{
  set Pin High;
```



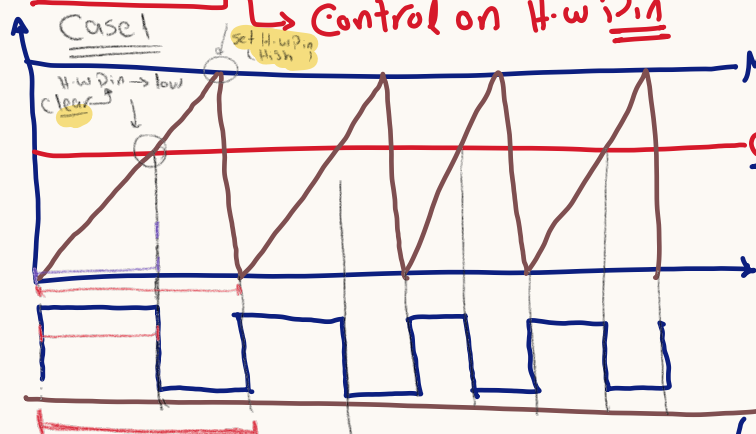
*

* Generate PWM by H.W

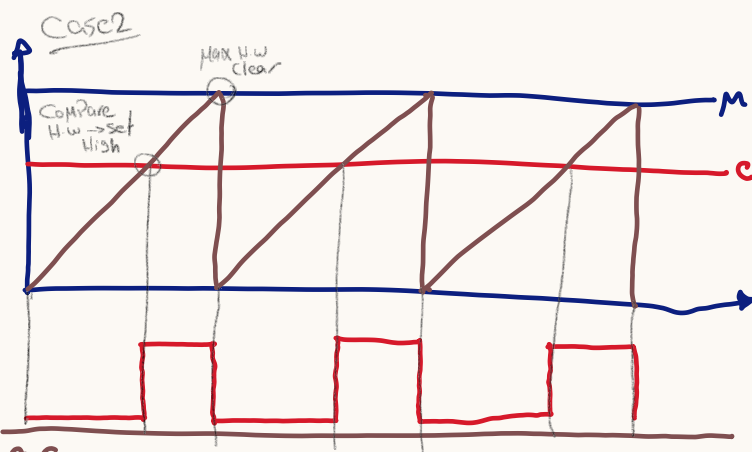
* Fast PWM

* Phase Correct PWM

* Fast PWM → Compare match
→ Control on H.W Pin



* Fast PWM



① Periodic time = over flow time.

② on time = compare time

③ $Duty_cycle = \frac{on}{Periodic}$ ← Compare value

$Duty_cycle = \frac{compare\ time}{over\ flow\ time}$

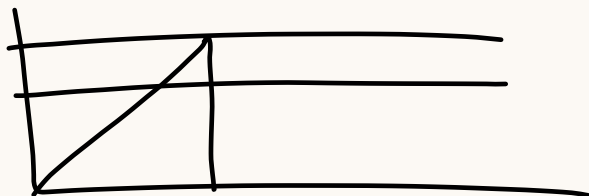
$Duty_cycle = \frac{Compare\ value \times clk\ time}{Max\ value \times clk\ time}$

Compare value

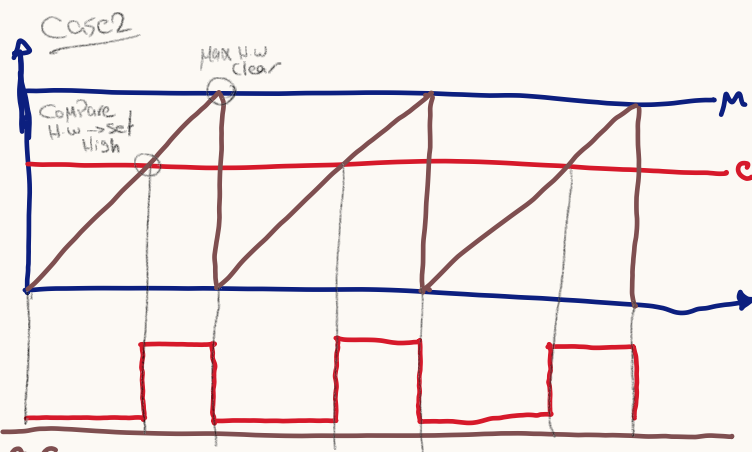
$= (Max\ value \times (Duty_cycle / 100))$

* Non Inverting

→ clear at compare
→ set at top (Bottom)



* Fast PWM



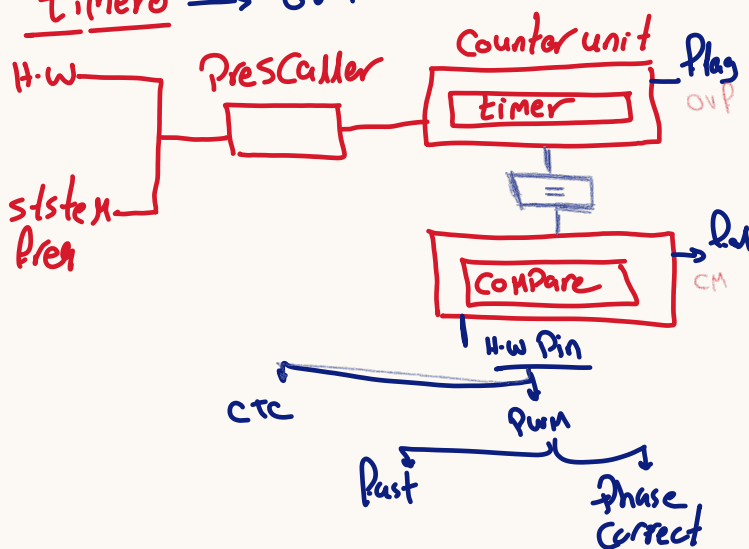
* Compare value =

$Max \times (1 - Duty\ cycle)$

* Inverting

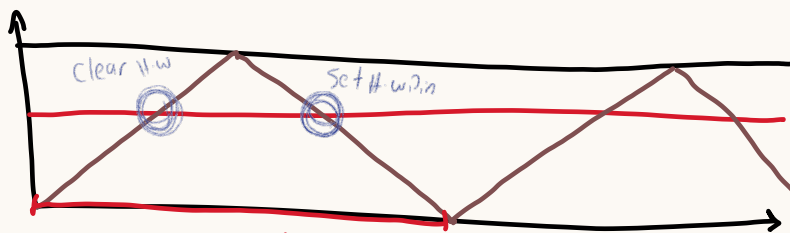
→ Compare set
→ Max clear

timer0 → 8bit.



* Phase Correct

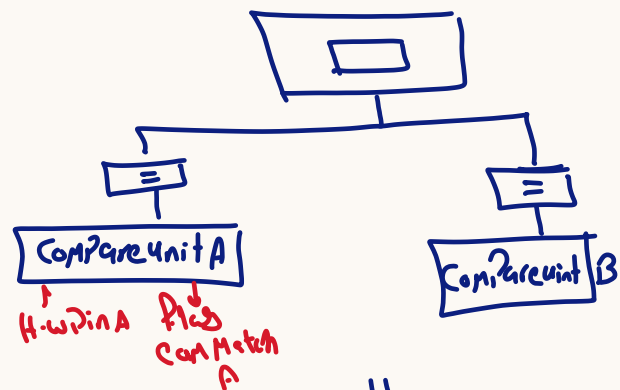
↳ large time ↳ small freq
↳ up Count | down Count



* Compare value = $\text{Max} \times \text{Duty cycle}$
↳ Inverting

* Non Inverting = $\text{Max} \times (1 - \text{Duty cycle})$

timer 1 → 16 :: bits



* timer 1 → Max → $\frac{11000}{20000}$
10000

timer 0 → ←